MotionVLA: Keyframe-and-Motion Memory for Long-Horizon Vision-Language-Action Policies

Anonymous Author(s)
Affiliation
Address
email

Abstract

Long-horizon robotic manipulation is non-Markovian: counting events, recovering object identity under occlusion, and reproducing demonstrated motions all require retaining many seconds of visual history while the policy runs at control rate within a fixed token budget. Recent benchmark evidence [Dai et al., 2026] shows that perceptual memory representations dominate motion-centric and time-sensitive tasks, but its concrete instantiations — uniform frame sampling and temporal-saliency-agnostic token dropping — are content-blind and degrade sharply as the memory budget tightens, while symbolic / language-subgoal memory cannot adequately represent low-level continuous motion. We introduce **MotionVLA**, a robot-specialized perceptual memory that decouples *what* the robot saw from *how* things moved between observations: a CLIP-derivative keyframe selector with delta-based per-keyframe TokenDrop carries sparse spatial evidence at semantic transitions, and a learned MotionEncoder, pretrained on unlabeled robot video via latent future-token prediction [Assran et al., 2023], summarizes each short window into a single compact motion token. The combined sequence is exposed to the $\pi_{0.5}$ backbone through a fixed-budget memory interface compatible with all three Memory-as-Context, -Modulator, and -Expert integration mechanisms used in the MME-VLA suite [Dai et al., 2026], enabling apples-to-apples comparison against uniform-frame-sampling and token-dropping baselines at matched memory budgets. Full quantitative evaluation on RoboMME is *in progress* (Sec. 4).

1 Introduction

Open-world robotic manipulation is rarely Markovian. A household robot that returns books to the right shelves must remember where each book originally came from; a robot asked to "press the button *exactly* when the moving cube reaches the target a specified number of times" must remember an event count and an event ordering; a robot reproducing a demonstrated trajectory must replay motion patterns it observed seconds earlier [Dai et al., 2026]. Modern vision-language-action (VLA) policies run at control rate, on a transformer with a fixed input window, so the central design question for memory-augmented manipulation is not *whether* to retain history but *which projection of it* fits inside that window without losing decision-relevant signal.

Existing memory designs fall into two families, each with limitations. *Symbolic* memory [Bu et al., 2025, Sridhar et al., 2025, Gemini Robotics Team et al., 2025] discretizes history into language subgoals; controlled evaluation [Dai et al., 2026] shows it under-specifies continuous motion. *Perceptual* memory retains raw visual tokens [Jang et al., 2025, Koo et al., 2025, Fang et al., 2025, Shi et al., 2026, Physical Intelligence, 2026] and is empirically stronger, but the two perceptual instantiations in that study — uniform frame sampling and saliency-agnostic token dropping — are content-blind and degrade steeply as the memory budget shrinks.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Robot videos have complementary structure: a few moments that genuinely matter (contact events, viewpoint shifts) and predictable motion in between. The right memory should mirror that: semantic keyframes for *what was seen and when*, and a compact learned summary for *how things moved*. MotionVLA addresses this gap with two complementary components. We train a MotionEncoder to predict future SigLIP latents in latent space, following Assran et al. [2023], forcing each motion token to encode exactly what content-blind subsampling discards.

Concretely, MotionVLA consists of: (1) a CLIP-derivative keyframe detector plus delta-based TokenDrop produces sparse but semantically anchored keyframe patches; (2) a MotionEncoder f_θ compresses each short window into one motion token (decoder discarded after pre-training); (3) the fixed-budget memory $\mathbf{M}_t$ plugs into the $\pi_{0.5}$ backbone via the same MaC / MaM / MaE interfaces as MME-VLA [Dai et al., 2026], enabling apples-to-apples comparison against content-blind baselines at matched budgets.

Contributions. The main contributions of MotionVLA are:

- A perceptual memory that couples CLIP-scored keyframes with delta-based TokenDrop and a learned MotionEncoder, decoupling spatial evidence from motion summaries in a fixed token budget.
- A latent prediction objective (following Assran et al. [2023]) with an auxiliary MotionDiT decoder (discarded at deployment) that forces each motion token to encode the information needed to bridge consecutive observations.
- A controlled evaluation on RoboMME [Dai et al., 2026]: our memory plugs into the MaC / MaM / MaE interfaces of MME-VLA on $\pi_{0.5}$, enabling matched-budget comparison against content-blind baselines (*evaluation in progress*).

2 Related Work

Memory-based Robotic Manipulation Models. History-dependent manipulation has motivated a diverse family of memory designs. Context-based approaches concatenate past visual tokens (ContextVLA [Jang et al., 2025]) or compress them recurrently (BC-RNN [Mandlekar et al., 2021]; Mamba-style: RoboMamba [Liu et al., 2024], MITL [Zhou et al., 2025]), but either pay linear attention cost or compress without selection. Explicit memory-bank methods retrieve on demand: Hamlet [Koo et al., 2025], MemoryVLA [Shi et al., 2026], and SAM2Act [Fang et al., 2025] cache visual or segmentation features; MEM [Physical Intelligence, 2026] organizes them at multiple temporal resolutions; BPP [Sobol Mark et al., 2026] explicitly selects key history frames. A third direction offloads memory to a VLM (UniVLA [Bu et al., 2025], MemER [Sridhar et al., 2025], Gemini-Robotics-1.5 [Gemini Robotics Team et al., 2025]), at the cost of modular inference and broken end-to-end optimization. These works differ in backbone, stored representation, and injection mechanism, making head-to-head comparison difficult — the gap our study targets.

Efficient Video Representation for Robotic Manipulation. Robot videos are heavily redundant, yet policies must retain seconds of history at control rate within a fixed token window. Prior work aggregates at the encoder side (ContextVLA [Jang et al., 2025], Hamlet [Koo et al., 2025], long-context diffusion [Torne et al., 2025]) or selects key frames (BPP [Sobol Mark et al., 2026], SAM2Act [Fang et al., 2025]). General video methods (TimeChat-Online [Yao et al., 2025], VideoLLM-online [Chen et al., 2024]) show that $\sim 80\%$ of tokens are redundant, but manipulation demands tighter budgets still. We combine delta-based TokenDrop over semantic keyframes with a MotionEncoder summary, decoupling sparse spatial evidence from short-range dynamics within a single fixed-budget representation.

3 Method

MotionVLA is a memory-augmented VLA policy that decouples *keyframe tokens* from *motion tokens* within a fixed budget. The following subsections describe its architecture (Sec. 3.1), memory generation (Sec. 3.2), and training (Sec. 3.3).

3.1 Model Architecture

MotionVLA is a memory-augmented vision-language-action (VLA) model built on top of the $\pi_{0.5}$ backbone. We initialize from the pre-trained backbone and fine-tune its experts end-to-end in Stage 2 (Sec. 3.3). The only components kept frozen throughout both stages are the SigLIP2 visual encoder and, after Stage 1, the MotionEncoder f_θ. We inject the per-episode memory sequence assembled in Sec. 3.2 as additional context tokens.

$\pi_{0.5}$ backbone. At history timestep t, the policy receives language tokens $\boldsymbol{\ell} \in \mathbb{R}^{N_\ell \times d}$ and multi-view image tokens $\mathbf{o}_t \in \mathbb{R}^{N_o \times d}$ extracted by a frozen SigLIP2 encoder. Following the π_0/$\pi_{0.5}$ multi-expert design [**??**], the backbone is composed of a *VLM expert* E_{vlm} that fuses language and vision and an *action expert* E_{act} that predicts an action chunk conditioned on the VLM features and a denoising timestep τ. Both experts are L-layer transformers connected through blockwise causal attention; the action expert is further conditioned on τ via AdaLN-Zero modulation [**?**]. We refer the reader to the original papers for the exact equations. Unlike the original $\pi_{0.5}$, which uses the VLM expert to also generate low-level language commands, we keep the VLM as a pure feature extractor; this leaves a clean interface at which memory can be injected.

Memory token block. To enable history-dependent reasoning, we augment the policy with an explicit memory state $\mathbf{M}_t$:

$$a_t \;\sim\; \pi(\cdot \mid l,\; \mathbf{o}_t,\; \mathbf{M}_t)\,, \tag{1}$$

where $\mathbf{M}_t$ is the length-B projection of the causal prefix $\mathcal{M}_t$ (Eq. 6), containing only observations and motion windows that have fully elapsed before step t. For fair comparison across designs, we allocate a fixed memory token budget B for all memory-based models: each representation produces at most B tokens; if fewer are available, the sequence is zero-padded. Thus all memory variants share the same interface

$$\mathbf{M}_t \;\in\; \mathbb{R}^{B \times d}. \tag{2}$$

We expose the memory sequence $\mathcal{M}$ produced in Sec. 3.2 to the backbone as this token block. The two-token-kind structure of $\mathcal{M}$ — TokenDropped keyframe patches and MotionEncoder motion tokens — is opaque to the backbone: from the policy's perspective $\mathbf{M}_t$ is just a length-B sequence of d-dimensional vectors carrying time tags.

Three integration mechanisms. We systematically study three places at which $\mathbf{M}_t$ can enter the backbone, trading parameter count, attention budget, and isolation of the pretrained VLM features against each other.

(1) Memory-as-Context (MaC). Memory tokens are concatenated with the standard image and language inputs of the VLM expert and jointly processed. This introduces no new parameters but lets memory compete with raw observation tokens for the same attention budget inside E_{vlm}.

(2) Memory-as-Modulator (MaM). The action latent at each layer cross-attends $\mathbf{M}_t$, and the result is mapped to per-feature scale/shift parameters through a zero-initialized MLP composed with AdaLN. The modulation is initialized to the identity, so fine-tuning starts from the pretrained $\pi_{0.5}$ behavior. MaM adds $\sim$80M parameters and leaves the VLM expert untouched.

(3) Memory-as-Expert (MaE). A dedicated third expert E_{mem} processes $\mathbf{M}_t$ through its own L transformer layers. Under an asymmetric blockwise causal mask, the VLM expert and memory expert use self-attention only and do not attend each other, while the action expert is the sole consumer that attends both as additional context. This preserves the pretrained $\pi_{0.5}$ representations exactly and allocates a separate transformer's worth of capacity ($\sim$190M parameters) to memory processing.

3.2 Memory Token Generation and Training

As the robot executes, the memory $\mathcal{M}_\tau$ grows online from the history of observations accumulated prior to policy step τ. It is a sparse, time-aligned sequence of two complementary token kinds: (i) *keyframe tokens*, which anchor the scene at semantic transitions, and (ii) *motion tokens*, fixed-stride summaries of short-range visual change produced by a learned MotionEncoder. Both kinds are encoded strictly from already-elapsed frames: keyframe tokens are committed as soon as a keyframe is detected, and a motion token m_t over window $[t,\, t{+}\Delta]$ becomes available only after step $t{+}\Delta$

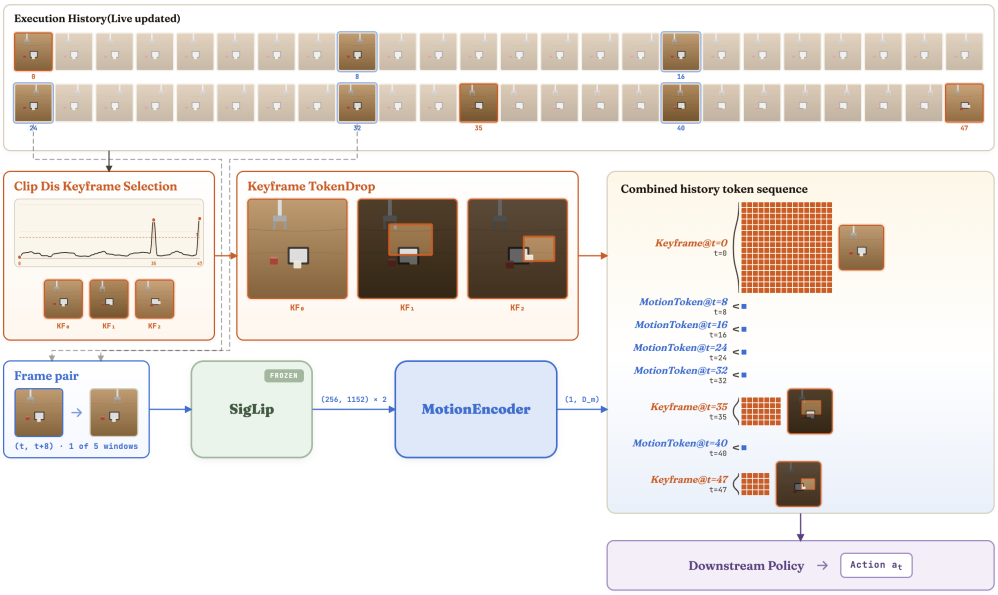

Figure 1: **Overall memory pipeline.** A CLIP encoder selects semantic keyframes from the robot trajectory; TokenDrop retains only the changed patches per keyframe. A learned MotionEncoder compresses each short frame-pair window into a single motion token via latent future prediction. The interleaved keyframe and motion tokens form a fixed-budget memory sequence $\mathbf{M}_t$ that conditions the downstream policy to predict action a_t.

has passed (Eq. 6). All encoding is performed by frozen perception backbones independently of the VLA; no gradient flows back into these encoders during VLA training.

Keyframe detection. Let $\{I_t\}_{t\geq 0} \subset \mathbb{R}^{3\times H\times W}$ denote the sequence of RGB frames observed during execution. Keyframes mark moments of meaningful scene change (object pickup, contact events, viewpoint shifts). Each frame is scored by the smoothed discrete derivative of a frozen CLIP encoder Φ_{CLIP}:

$$d_t \;=\; \left\|\Phi_{\text{CLIP}}(I_t) - \Phi_{\text{CLIP}}(I_{t-1})\right\|_2, \qquad \tilde{d}_t \;=\; (k_{\text{rbf}} * d)_t, \tag{3}$$

where k_{rbf} is a small RBF kernel that suppresses frame-rate jitter. Keyframes are local maxima of $\tilde{d}_t$ exceeding a threshold τ, with the first frame $t = 0$ always retained as an anchor. The detector is purely deterministic preprocessing and has no learnable parameters; τ is a single hyperparameter chosen so that the average inter-keyframe gap is ~ 32 frames.

Keyframe TokenDrop. Storing the full S-token grid for every keyframe is wasteful: most of the image (background, robot base, table) repeats across keyframes. For every keyframe k_i with $i > 0$ we retain only the patches that actually changed relative to its *previous* keyframe k_{i-1}. Concretely, on the 16×16 patch grid we mark a patch as *significant* if its mean per-pixel absolute difference between I_{k_i} and $I_{k_{i-1}}$ exceeds δ, and we keep the union of all significant patches together with the patches lying inside their axis-aligned bounding rectangle (the latter preserves spatial contiguity for downstream attention). The first keyframe k_0 is always kept in full (S tokens); each subsequent keyframe contributes $|\mathcal{P}_i| \leq S$ tokens. TokenDrop is also parameter-free.

Motion tokens. Let $\Phi_{\text{SigLIP}} : \mathbb{R}^{3\times H\times W} \to \mathbb{R}^{S\times E}$ ($S = 256$ patches, $E = 1152$) be the frozen SigLIP tokenizer, and let $x_t = \Phi_{\text{SigLIP}}(I_t) \in \mathbb{R}^{S\times E}$; $\{s_t\}$ denotes the proprioceptive state sequence. Independently of keyframe placement, we tile the video with non-overlapping windows of fixed stride Δ (we use $\Delta = 8$). Following the latent prediction approach of Assran et al. [2023], for each window $[t,\, t+\Delta]$ a trainable *MotionEncoder* f_θ compresses the visual change into M latent tokens $m_t \in \mathbb{R}^{M\times D_m}$:

$$m_t \;=\; f_\theta(\, x_t,\; x_{t+\Delta},\; \Delta s_t\,)\,, \qquad \Delta s_t = s_{t+\Delta} - s_t. \tag{4}$$

Internally, f_θ (i) computes the patchwise visual delta $\Delta x = x_{t+\Delta} - x_t \in \mathbb{R}^{S\times E}$, (ii) spatially average-pools Δx to a single global delta token in $\mathbb{R}^{1\times E}$ and projects it to the encoder width H, (iii) projects Δs_t to H and adds it to the pooled delta token (this is the only place proprioception enters the memory pipeline), (iv) appends M learned read-out queries at the tail, and (v) runs a small causal transformer (RMSNorm, 1D-RoPE causal self-attention, SwiGLU FFN) so that each query attends to the fused delta token before it. The last M positions are projected to motion dimension D_m and constitute m_t. Since m_t requires both endpoints of window $[t,\ t{+}\Delta]$, it becomes causally available only at step $t{+}\Delta$.

Memory sequence assembly. MotionVLA assembles the final memory for an episode as the time-ordered concatenation

$$\mathcal{M} \;=\; \big[\, x^{\text{full}}_{k_0},\ m_0,\ m_\Delta,\ \ldots,\ x^{\text{TD}}_{k_i},\ \ldots \,\big], \tag{5}$$

where each entry carries its source timestamp as a positional tag. At policy step τ, the policy receives the causally valid prefix

$$\mathcal{M}_\tau \;=\; \big\{\, x_{k_j} : k_j \le \tau \,\big\} \;\cup\; \big\{\, m_t : t+\Delta \le \tau \,\big\}, \tag{6}$$

i.e. keyframe tokens whose frame index is at most τ, and motion tokens whose entire window $[t,\ t{+}\Delta]$ has elapsed by τ. The sequence $\mathcal{M}_\tau$ grows monotonically: at each new step the pipeline appends newly completed tokens without recomputing earlier ones. Keyframe selection and TokenDrop are training-free; the only learnable parameters in the memory pipeline are those of the MotionEncoder f_θ, which is pre-trained in Stage 1 of Sec. 3.3 and then frozen. The trained f_θ, together with the parameter-free keyframe and TokenDrop modules, is what ultimately writes into $\mathcal{M}$.

3.3 Training Objective

The full system has two distinct loss surfaces, optimized in two stages: a latent future-prediction loss $\mathcal{L}_{\text{pred}}$ that trains the MotionEncoder f_θ of Sec. 3.2, and a flow-matching loss $\mathcal{L}_{\text{FM}}$ that trains the VLA backbone together with the integration module.

Latent future-prediction loss for f_θ. Following Assran et al. [2023], we train f_θ in latent space jointly with an auxiliary *MotionDiT* decoder g_ϕ (Fig. 2): a conditional-diffusion decoder that regresses the SigLIP tokens of the future frame from the current tokens and the motion code. Given $(x_t, x_{t+\Delta})$, we sample a diffusion-style timestep $k \sim \mathcal{U}\{1,\ldots,K\}$ and ask g_ϕ to predict the (token-normalized) future tokens from the current tokens, the motion code, and k:

$$\hat{x}^{(k)}_{t+\Delta} \;=\; g_\phi(\, x_t,\ m_t,\ k\,), \qquad m_t = f_\theta(x_t, x_{t+\Delta}, \Delta s_t), \tag{7}$$

$$\mathcal{L}_{\text{pred}} \;=\; \mathbb{E}_{t,\,k}\Big[\, \big\|\hat{x}^{(k)}_{t+\Delta} \;-\; \text{TokenNorm}(x_{t+\Delta})\big\|_2^2 \,\Big]. \tag{8}$$

TokenNorm is a per-feature standardization applied to the target SigLIP tokens to stabilize the regression target. g_ϕ is a small DiT with AdaLN-zero timestep modulation and a tied gated input/output projection initialized to zero, so at $k{=}0$ the decoder acts as a normalized identity copy of x_t and only learns residual corrections driven by m_t. This zero-initialized residual head is the sole reason the loss is stable when f_θ is randomly initialized: gradients flow through g_ϕ into f_θ only via the correction term, forcing m_t to encode exactly the information needed to bridge $x_t \to x_{t+\Delta}$. Crucially, g_ϕ exists only during pre-training: at deployment the robot has already executed both endpoints of every motion window, so memory tokens are produced by a single forward pass through Φ_{SigLIP} and f_θ; no decoding is performed and g_ϕ is discarded.

Flow-matching action loss. Following $\pi_{0.5}$, the policy predicts an H-step action chunk $\mathbf{A}_t \in \mathbb{R}^{H\times D}$ via conditional flow matching: the action expert parameterizes a velocity field $\mathbf{v}_\theta(\mathbf{A}^\tau_t, \tau, \mathbf{h}_t)$ trained to match the noise-to-data transport direction,

$$\mathcal{L}_{\text{FM}} \;=\; \mathbb{E}\Big[\, \big\|\mathbf{v}_\theta\big(\mathbf{A}^\tau_t, \tau, \mathbf{h}_t\big) \;-\; \big(\mathbf{A}^{\text{noise}}_t - \mathbf{A}^{\text{data}}_t\big)\big\|_2^2 \,\Big], \tag{9}$$

where the context features $\mathbf{h}_t$ are the layer-L outputs of the integrated backbone and therefore depend on the chosen integration mechanism (MaC, MaM, or MaE). At inference the action chunk is generated by integrating $\mathbf{v}_\theta$ from $\tau{=}1$ to $\tau{=}0$ through a small number of Euler steps.

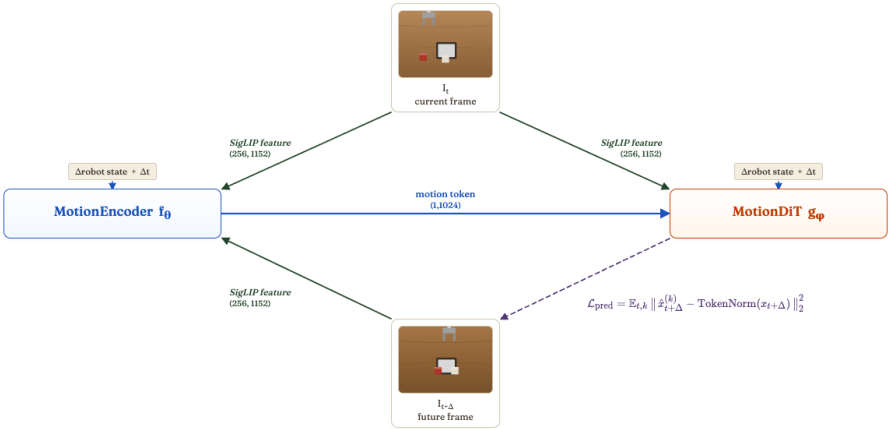

Figure 2: **MotionEncoder training mechanism.** The MotionEncoder f_θ and the auxiliary MotionDiT g_ϕ are trained jointly via JEPA-style latent prediction. Both modules receive SigLIP features from the current frame I_t; f_θ additionally takes the future frame $I_{t+\Delta}$ and the proprioceptive difference Δs_t to produce a compact motion token. g_ϕ then predicts the token-normalized future-frame features from the current tokens and the motion token, providing a latent prediction signal that trains f_θ. After pre-training, g_ϕ is discarded and f_θ is frozen.

Two-stage training. The two losses are optimized sequentially, not jointly, because the memory pipeline encodes a task-agnostic visual-dynamics prior whereas the integration module is task-specific.

Stage 1: Memory-pipeline pre-training. On unlabeled robot videos we jointly train the MotionEncoder f_θ and the auxiliary MotionDiT decoder g_ϕ to minimize $\mathcal{L}_{\text{pred}}$. The SigLIP and CLIP backbones, the keyframe detector, and the TokenDrop module are training-free throughout. After Stage 1, g_ϕ is discarded and f_θ is frozen.

Stage 2: VLA fine-tuning. For every demonstration episode we pre-compute and cache the full memory sequence $\mathcal{M}$ using the frozen pipeline of Stage 1, together with SigLIP2 visual tokens. At each training step t the causally valid prefix $\mathcal{M}_t$ (Eq. 6) is projected to $\mathbf{M}_t \in \mathbb{R}^{B\times d}$ and fed to the chosen integration module; the VLM expert, action expert, and integration module are trained end-to-end on $\mathcal{L}_{\text{FM}}$. No proprioceptive state enters the backbone in Stage 2 — proprioception only participates in Sec. 3.2 when forming motion tokens.

This decoupling is what makes the same memory pipeline reusable across all three integration variants and across downstream tasks: Stage 1 is run once; Stage 2 is repeated for each (task, integration) configuration.

3.4 Implementation Details

Memory budget and compression. We cap the keyframe-token memory at $B = 2000$ tokens of width $d = 1024$; the MotionEncoder contributes up to 125 motion tokens separately, giving a combined budget of $2000{+}125$, padding with zeros when shorter. Compared with the naive flattened representation of $T \cdot S$ patch tokens per episode, this is a $\sim 250\times$ reduction at $T \approx 2000$, while preserving both the long-range keyframe anchors (the first keyframe contributes a full $S = 256$ tokens) and short-range dynamics ($M = 1$ motion token per $\Delta = 16$-frame window).

4 Experiments

Our evaluation of MotionVLA is structured around three research questions. **RQ1 (memory content).** Under a fixed token budget and a fixed integration mechanism, do content-aware perceptual memories (CLIP keyframes, TokenDrop, learned motion tokens) preserve more decision-relevant signal than the content-blind subsampling baselines used in MME-VLA [Dai et al., 2026]? **RQ2 (budget**

sensitivity). How does each memory variant degrade as the memory budget B shrinks — gracefully, or with a cliff? **RQ3 (integration).** Among Memory-as-Context, Memory-as-Modulation, and Memory-as-Expert, which integration mechanism best exploits a given memory representation? At the time of submission, the budget-sensitivity component of RQ2 has finished for both content-blind baselines, while the keyframe-based configurations and the cross-backbone study addressing RQ1 and RQ3 are still running; we report what is currently available and explicitly mark all unfinished cells as *ongoing* (Sec. 4.2).

4.1 Experimental Setup

Benchmark. We evaluate on RoboMME [Dai et al., 2026], a long-horizon, history-dependent manipulation suite spanning four memory categories (temporal, spatial, object, procedural). It provides matched train/eval splits, a unified scoring protocol, and the MME-VLA reference policies, making it the natural testbed for controlled memory-content comparisons. Compared methods are detailed in Sec. 4.2.

Metric. Following RoboMME [Dai et al., 2026], we report task Success Rate (%) averaged over the benchmark's evaluation episodes; higher is better.

Training and inference. We follow the two-stage recipe of Sec. 3.3: Stage 1 pre-trains the MotionEncoder f_θ once on unlabeled robot video with $\mathcal{L}_{\text{pred}}$, after which f_θ is frozen and reused across every (memory, integration, backbone) configuration; Stage 2 fine-tunes the policy with $\mathcal{L}_{\text{FM}}$ for 80k steps under the optimizer and hardware setup described in Sec. 3.4. At inference the memory pipeline runs strictly causally (Eq. 6): only motion windows whose right endpoint has elapsed before the current step contribute to $\mathbf{M}_t$.

4.2 Main Results

This subsection collects the results currently available against the memory-content question (RQ1) and the budget-sensitivity question (RQ2). The completed runs cover the two content-blind perceptual baselines (*Even Frame Sampling* and *Even TokenDropping*) across the full budget sweep $B \in \{1024, 512, 256, 128, 64\}$ under MaM; the keyframe-based configurations and the cross-backbone study are still running and appear as *ongoing* entries in Tab. 1.

Baselines. All configurations share the $\pi_{0.5}$ backbone and MaM integration, so differences are attributable to memory *content*:

- **MME-VLA Clip-Keyframe**: retains every token of selected keyframes without compression ($\sim$8000+ tokens; upper bound).
- **MME-VLA Clip-Keyframe + TokenDrop**: adds delta-based patch retention, compressing to a fixed budget of 2000 tokens.
- **MotionVLA (Ours) — Clip-KF + TokenDrop + Motion**: further appends 125 motion tokens, yielding $2000+125$ total.
- **MME-VLA Even Frame Sampling (EFS)**: uniformly spaced frames with pooled tokens, swept over $B \in \{1024, 512, 256, 128, 64\}$.
- **MME-VLA Even TokenDropping (ETD)**: saliency-agnostic patch dropping, swept over the same five budgets.
- **Alternative backbones** (OpenVLA, Qwen2.5-VL): same memory pipeline on different VLA families, probing transferability.

Findings on the content-blind baselines. Two observations follow from Tab. 1. *(i)* EFS outperforms ETD at every budget ($+5.7$ pt at $B=1024$, widening to $+12.3$ pt at $B=64$): full frames carry more usable signal than the same number of arbitrarily selected patches. *(ii)* EFS degrades gradually (15.45 pt over the full sweep), while ETD collapses sharply between $B=128$ and $B=64$ (14.65 pt single-step drop), suggesting content-blind patch removal crosses a representational threshold at tight budgets. Whether content-aware selection closes this gap is the central remaining question; those rows are still running.

Table 1: Main results on RoboMME. We compare our *Clip-Keyframe* + *TokenDrop* + *Motion* memory against keyframe-only / token-drop variants and the FrameSamp / TokenDrop budget sweeps, all under the *Memory-as-Modulation* integration mechanism. Success Rate is reported in %; *ongoing* denotes runs whose evaluation has not yet finished.

Method	Integration	Memory Budget	Success Rate (%)
MotionVLA (Ours): Clip-KF + TokenDrop + Motion	Modulation	2000+125	*ongoing*
MME-VLA Clip-Keyframe	Modulation	8000+	*ongoing*
MME-VLA Clip-Keyframe + TokenDrop	Modulation	2000	*ongoing*
MME-VLA Even Frame Sampling	Modulation	1024	45.87
	Modulation	512	44.51
	Modulation	256	42.90
	Modulation	128	36.54
	Modulation	64	30.42
MME-VLA Even TokenDropping	Modulation	1024	40.17
	Modulation	512	38.02
	Modulation	256	36.36
	Modulation	128	32.76
	Modulation	64	18.11
Other backbones (OpenVLA, Qwen2.5-VL)	—	—	*ongoing*

4.3 Ablation Studies

A full ablation over (i) keyframe selection (CLIP vs. uniform), (ii) TokenDrop on/off, (iii) motion tokens on/off, (iv) integration mechanism (MaC/MaM/MaE), and (v) backbone (OpenVLA, Qwen2.5-VL) is currently running; results will replace the *ongoing* cells in Tab. 1.

4.4 Analysis

These findings are consistent with our motivation in Sec. 1: content-blind patch removal crosses a representational threshold at tight budgets, while frame-level subsampling degrades more gracefully by always delivering a complete spatial layout. The analysis is restricted to the two completed sweeps; per-task breakdowns and failure cases will be reported once the *ongoing* runs finish.

5 Conclusion

We proposed MotionVLA (Sec. 3), a robot-specialized perceptual memory that decouples semantic keyframes from learned short-range motion summaries within a fixed token budget, exposed to the $\pi_{0.5}$ backbone through three interchangeable interfaces (MaC / MaM / MaE). The completed budget sweeps establish that the two content-blind perceptual variants of MME-VLA degrade in qualitatively different ways: *Even Frame Sampling* loses 15.45 pt nearly linearly across $B \in [1024, 64]$, while *Even TokenDropping* loses 22.06 pt with a sharp 14.65-pt cliff at $B = 64$, confirming that which patches are retained matters most when the budget is tightest — the regime our content-aware design targets.

Limitations and future work. Quantitative evaluation is incomplete at submission; the keyframe-based configurations, integration-mechanism comparison (RQ3), and cross-backbone study are still running, with full results to appear in the camera-ready version. Future directions for MotionVLA include (i) online keyframe reassignment as episodes evolve; (ii) hybrid perceptual-plus-language memories combining our tokens with VLM-driven subgoal representations; and (iii) validation on real hardware where sensor noise will further stress the keyframe detector and MotionEncoder.

References

M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023.

Q. Bu, Y. Yang, J. Cai, S. Gao, G. Ren, M. Yao, P. Luo, and H. Li. Learning to act anywhere with task-centric latent actions. In *Robotics: Science and Systems (RSS)*, 2025.

J. Chen, Z. Lv, S. Wu, K. Q. Lin, C. Song, D. Gao, J.-W. Liu, Z. Gao, D. Mao, and M. Z. Shou. VideoLLM-online: Online video large language model for streaming video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18407–18418, 2024.

Y. Dai, H. Fu, J. Lee, Y. Liu, H. Zhang, J. Yang, C. Finn, N. Fazeli, and J. Chai. RoboMME: Benchmarking and understanding memory for robotic generalist policies. *arXiv preprint arXiv:2603.04639*, 2026.

H. Fang, M. Grotz, W. Pumacay, Y. R. Wang, D. Fox, R. Krishna, and J. Duan. Sam2Act: Integrating visual foundation model with a memory architecture for robotic manipulation. In *International Conference on Machine Learning (ICML)*, 2025.

Gemini Robotics Team, A. Abdolmaleki, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. Gonzalez Arenas, A. Balakrishna, N. Batchelor, A. Bewley, J. Bingham, et al. Gemini robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer. *arXiv preprint arXiv:2510.03342*, 2025.

H. Jang, S. Yu, H. Kwon, H. Jeon, Y. Seo, and J. Shin. ContextVLA: Vision-language-action model with amortized multi-frame context. *arXiv preprint arXiv:2510.04246*, 2025.

M. Koo, D. Choi, T. Kim, K. Lee, C. Kim, Y. Seo, and J. Shin. Hamlet: Switch your vision-language-action model into a history-aware policy. *arXiv preprint arXiv:2510.00695*, 2025.

J. Liu, M. Liu, Z. Wang, P. An, X. Li, K. Zhou, S. Yang, R. Zhang, Y. Guo, and S. Zhang. RoboMamba: Efficient vision-language-action model for robotic reasoning and manipulation. *Advances in Neural Information Processing Systems*, 37:40085–40110, 2024.

A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. *arXiv preprint arXiv:2108.03298*, 2021.

Physical Intelligence. MEM: Multi-scale embodied memory for vision language action models, 2026.

H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, E. Zhou, H. Fan, X. Zhang, and G. Huang. MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2026.

N. Sobol Mark et al. BPP: Long-context robot imitation learning by focusing on key history frames, 2026.

A. Sridhar, J. Pan, S. Sharma, and C. Finn. MemER: Scaling up memory for robot control via experience retrieval. *arXiv preprint arXiv:2510.20328*, 2025.

M. Torne, A. Tang, Y. Liu, and C. Finn. Learning long-context diffusion policies via past-token prediction. *arXiv preprint arXiv:2505.09561*, 2025.

L. Yao, Y. Li, Y. Wei, L. Li, S. Ren, Y. Liu, K. Ouyang, L. Wang, S. Li, S. Li, et al. TimeChat-Online: 80% visual tokens are naturally redundant in streaming videos. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 10807–10816, 2025.

Y. Zhou, Y. Lin, F. Peng, J. Chen, K. Huang, H. Yang, and Z. Yin. MTIL: Encoding full history with mamba for temporal imitation learning. *IEEE Robotics and Automation Letters*, 2025.